

# BTC VOLATILITY RESEARCH

## Cours complet d'architecture logicielle

Pipeline options Deribit → IV → calibration SVI / Heston / Merton

Ce que fait chaque module, comment les données circulent, les concepts financiers sous-jacents, et ou optimiser.

### Au sommaire

1. Vue d'ensemble : à quoi sert le projet
2. Le flux de données (source → traitement → sortie)
3. Rappels financiers : IV, smile, SVI, Heston, Merton
4. Visite guidée dossier par dossier
5. Le chemin critique du calcul de volatilité
6. Ou et comment optimiser (perf + maintenabilité)

Genere automatiquement à partir du code source du depot.

# 1. Vue d'ensemble : a quoi sert le projet

Le projet reconstruit et modelise la surface de volatilité implicite des options Bitcoin cotees sur Deribit. Concretement, il part de cotations brutes d'options (prix bid/ask/mark, strikes, maturites) stockees dans une base PostgreSQL, en deduit la volatilité implicite de chaque option, puis ajuste trois modeles de reference sur cette surface pour la resumer par quelques parametres interpretables.

## Les trois modeles calibres

- **SVI** : modele parametrique STATIQUE du smile, une maturite a la fois. Formule fermee (aucune EDP), tres rapide. Sert de baseline descriptive du smile.
- **Heston** : modele a volatilité stochastique. Prix calcules via QuantLib (moteur analytique), calibre maturite par maturite. Capture le smile via la correlation spot/vol et la vol-of-vol.
- **Merton** : modele a sauts (jump-diffusion). Prix = serie de Poisson x Black-Scholes. Calibre GLOBALEMENT sur toute la surface d'un coup. Capture les ailes (smile court terme) grace aux sauts.

Chaque modele produit : des parametres calibres, des metriques d'erreur (RMSE en points de vol, par zone de moneyness), des graphiques de smile, et une surface 3D interactive (Plotly). Tout est ecrit dans outputs/reports (CSV) et outputs/figures (PNG + HTML).

### Glossaire express

IV = volatilité implicite : la vol qui, injectee dans Black-Scholes, redonne le prix de marche.  
Smile = courbe IV en fonction du strike, a maturite fixee. Surface = smile empile sur toutes les maturites. Moneyness (log) =  $\ln(K / \text{forward})$ , 0 = a la monnaie (ATM).

## 2. Le flux de donnees

Le projet est un pipeline lineaire. Chaque etage a une responsabilite unique et passe un DataFrame enrichi a l'etage suivant. C'est la colonne vertebrale a garder en tete pour lire n'importe quel fichier du depot.

```
PostgreSQL (table btc_options, snapshots Deribit)
|
v data/loader.py -> load_snapshot()
DataFrame brut (prix, strikes, maturites, mark_iv...)
|
v data/panel.py -> build_market_panel()
Panel enrichi (forward, log_moneyness, iv_mid inversee BS,
              iv_used, filtres qualite, slice_id)
|
+-----+-----+
v               v               v
models/svi      models/heston    models/merton
calibrate_*()   calibrate_*()    calibrate_global()
(par tranche)   (par tranche)    (surface entiere)
|               |               |
v               v               v
surfaces/*_surface.py -> grille (k, T) -> IV
|
v analysis/* + surfaces/plots.py
CSV (outputs/reports) + PNG/HTML (outputs/figures)
```

### Trois granularites de calibration a distinguer

- SVI et Heston calibrent UNE TRANCHE (une maturite) a la fois : boucle groupby('slice\_id') -> un jeu de parametres par maturite. C'est calibrate\_all\_slices().
- Merton calibre UN SEUL jeu de parametres sur TOUT le panel simultanement : calibrate\_global(). Les erreurs sont ensuite redecoupees par maturite a posteriori (build\_slice\_fits).
- Cette difference explique pourquoi Merton a un pricer vectorise sur tout (S, K, T) alors que Heston boucle strike par strike.

## 3. Rappels financiers indispensables

Cette section explique les objets mathématiques manipulés, sans lesquels le code paraît arbitraire. Rien à coder ici : c'est la clé de lecture.

### 3.1 Black-Scholes et volatilité implicite

Black-Scholes donne le prix d'une option à partir de  $(S, K, T, r, q, \sigma)$ . Le seul paramètre non observable est  $\sigma$ . La volatilité IMPLICITE est l'opération inverse : étant donné un prix de marché, on cherche le  $\sigma$  qui le reproduit. Il n'existe pas de formule fermée → on inverse numériquement (Newton-Raphson).

Dans le code : `iv/black_scholes.py` contient les deux mondes. Les fonctions scalaires (`bs_call_price`, `bs_vega`...) pour un point, et les versions `'_vec'` vectorisées NumPy pour des tableaux entiers. `implied_volatility()` fait l'inversion Newton vectorisée : toutes les options convergent en parallèle, avec un masque `'active'` qui retire celles déjà convergées à chaque itération.

```
# Cle de l'inversion (black_scholes.py) : le vega est la dérivée
# du prix par rapport à sigma, donc le pas de Newton est
#     sigma <- sigma - (prix_modele - prix_marche) / vega
# Vectorise : un seul appel traite N options à la fois.
```

### 3.2 Le smile et la moneyness

Si Black-Scholes était exact,  $\sigma$  serait constant pour tous les strikes. En pratique, l'IV forme une courbe en U (le 'smile') : les options loin de la monnaie ont une IV plus élevée. On représente le smile en log-moneyness  $k = \ln(K/\text{forward})$ , ce qui centre le ATM en  $k = 0$  et rend les maturités comparables. La zone ATM ( $|k|$  petit) est la plus liquide et la plus fiable ; les 'ailes' (left/right wing) sont plus bruitées.

### 3.3 SVI (Stochastic Volatility Inspired)

SVI (Gatheral) paramètre directement la VARIANCE TOTALE  $w(k) = \sigma^2 * T$  par une formule à 5 paramètres ( $a, b, \rho, m, \sigma$ ). Ce n'est pas un modèle de dynamique : c'est une courbe flexible qui épouse le smile d'une maturité donnée. Avantage : formule fermée, calibration quasi instantanée. La condition `'butterfly_ok'` garantit l'absence d'arbitrage papillon (densité positive).

```
w(k) = a + b * ( rho*(k-m) + sqrt((k-m)^2 + sigma^2) )
IV(k) = sqrt( w(k) / T )
# fichier : models/svi/formula.py (pur NumPy, aucune boucle)
```

### 3.4 Heston (volatilité stochastique)

Heston suppose que la variance elle-même est un processus aléatoire (retour à la moyenne  $\kappa$  vers  $\theta$ , vol-of-vol  $\sigma$ , corrélation  $\rho$  avec le spot, variance initiale  $v_0$ ). La corrélation  $\rho$  négative génère naturellement le skew observé sur le marché. Il n'y a pas de formule fermée simple pour le prix → on passe par la fonction caractéristique et une intégration. Ici le projet délègue ce calcul à QuantLib (AnalyticHestonEngine), plus robuste. La contrainte de Feller ( $2*\kappa*\theta > \sigma^2$ ) garde la variance positive.

### 3.5 Merton (jump-diffusion)

Merton ajoute a la diffusion Black-Scholes des SAUTS discrets d'intensite  $\lambda$ , de taille log-normale ( $\mu_{\text{jump}}$ ,  $\sigma_{\text{jump}}$ ). Le prix est une somme ponderee (poids de Poisson) de prix Black-Scholes conditionnels au nombre de sauts. Les sauts creent le smile prononce a court terme que la diffusion pure ne peut pas produire.

```
prix = somme_{n=0}^{inf} P(N=n) * BS( S_n, K, T, sigma_n )  
avec P(N=n) = exp(-lam*T) (lam*T)^n / n!  
# la serie converge vite -> tronquee a 60 termes (pricer.py)
```

## 4. Visite guidée, dossier par dossier

Pour chaque module : son rôle, les fonctions clés, et un commentaire sur la qualité / les pistes d'amélioration.

### 4.1 config.py — la configuration

Charge la configuration depuis `configs/default.yaml` et les variables d'environnement (`.env`). Tout est en dataclasses frozen (immuables) : `MarketConfig` (taux, filtres de maturité/spread), `CalibrationConfig` (optimiseur, poids, pénalité), et les bornes par modèle (`HestonBounds`, `SVIBounds`, `MertonBounds`). `load_config()` fusionne YAML + defaults.

- `load_config()` : point d'entrée unique, retourne un `AppConfig` complet.
- `_merge_yaml()` : fusion récursive YAML/defaults (override propre).
- `merton_bounds` : propriété de compat pour du code/tests legacy.

### 4.2 data/ — accès et préparation des données

#### loader.py

I/O PostgreSQL pure. `get_connection()` ouvre la connexion Neon ; `load_snapshot()` lit toutes les lignes d'une date ; `latest_snapshot_date()` / `list_snapshot_dates()` pour naviguer. Rien de calculatoire ici : c'est volontairement une frontière mince.

#### panel.py (le cœur de la préparation)

`build_market_panel()` transforme le snapshot brut en panel exploitable : calcule le forward, le `log_moneyness`, convertit les primes BTC en USD, inverse l'IV mid via Black-Scholes, choisit `iv_used` (mid si fiable, sinon mark), applique les filtres qualité (maturité, open interest, spread, options fantômes), et pose `slice_id` (= la maturité). Entièrement vectorise pandas/NumPy.

#### Point de lecture

`iv_used` est LA colonne centrale : toutes les calibrations l'utilisent comme cible. Comprendre comment elle est construite (mid inversée, contrôlée contre mark, filtrée) c'est comprendre la fiabilité de tout ce qui suit.

### 4.3 iv/ — Black-Scholes et conventions

`black_scholes.py` : prix call/put, vega, forward, en versions scalaire ET vectorisée, plus l'inversion IV Newton-Raphson vectorisée. `conventions.py` : `relative_spread()` (mesure de liquidité) et `apply_smile_convention()` (garde les OTM par défaut, ou calls/puts/tous selon config).

### 4.4 models/calibration/ — briques PARTAGEES

Sous-package transverse, factorise ce qui est commun aux trois modèles. C'est le bon réflexe DRY du projet.

- `errors.py` : `iv_rmse`, `iv_mae`, `iv_sse`, `iv_weighted_rmse`, `iv_error_variance`.
- `filters.py` : `quality_filter()` (bornes IV et maturité communes).
- `results.py` : dataclasses `SliceFitResult` et `GlobalCalibrationResult` (générique).
- `slice_fits.py` : `build_slice_fits()` redécoupe un fit GLOBAL par maturité.
- `slice_metrics.py` : `slice_iv_diagnostics()` -> RMSE/MAE + RMSE par zone (ATM/ailes).

## 4.5 models/calibration\_weights.py — ponderations

Definit les poids  $w_i$  de la fonction objectif : vega (donne du poids aux options sensibles a la vol), liquidite (racine de l'open interest), volume. `calibration_weights (v1)`, `calibration_weights_v2`, `vega_only_weights`, `volume_only_weights`. `build_panel_weights()` assemble les poids tranche par tranche sur l'ordre du panel.

### Amelioration reperee

`_vega_vector()` calcule le vega dans une boucle Python (itertuples + `bs_vega` scalaire) alors que `bs_vega_vec()` existe deja, vectorise, dans le meme depot. Remplacement direct = 10 a 30x plus rapide sur cette fonction, zero changement de resultat.

## 4.6 models/svi/ — baseline smile

- **params.py** : `SVIPParams` (a,b,rho,m,sigma) + `butterfly_ok()` (non-arbitrage).
- **formula.py** : `svi_total_variance()` et `svi_iv_from_log_moneyness()` (closed-form).
- **calibrate.py** : `calibrate_slice()` minimise l'erreur ponderee sur une maturite ; `calibrate_all_slices()` boucle sur les tranches.

## 4.7 models/heston/ — volatilité stochastique

- **params.py** : `HestonParams` + `feller_satisfied()`.
- **pricer.py** : prix via QuantLib. `heston_iv_grid()` price chaque strike puis inverse.
- **calibrate.py** : `calibrate_slice()` / `calibrate_all_slices()` par maturite.
- **characteristic.py** : fonction caracteristique de Heston... JAMAIS importee (code mort).

### Deux ameliorations majeures ici

(1) `heston_iv_grid` reconstruit un moteur QuantLib complet a CHAQUE strike, dans la boucle d'optimisation : le construire une fois par tranche donne un gain 3-8x sans risque. (2) `characteristic.py` suggere un pricer Fourier commence puis abandonne : soit le supprimer, soit le finir (integration COS vectorisee) pour remplacer QuantLib.

## 4.8 models/merton/ — jump-diffusion

- **params.py** : `MertonParams` + `jump_compensation()` + `is_valid()`.
- **pricer.py** : `merton_option_prices()` vectorise sur (S,K,T) ; boucle Poisson (60 termes) ou chaque terme est vectorise. `merton_iv_panel()` price puis inverse tout le panel.
- **calibrate.py** : `calibrate_global()` -> un seul jeu de parametres sur toute la surface.
- **weight\_schemes.py** : registre extensible uniform/vega/volume (pattern propre).

### Un exemple a garder

Le pricer Merton est le modele de vectorisation du depot : la seule boucle porte sur l'ordre de la serie de Poisson (borne theorique), pas sur les donnees. C'est ce qu'il faut viser partout ailleurs.

## 4.9 surfaces/ — reconstruction de surface

- **surface.py** : interpolation griddata brute du panel (surface de marche).
- **smile.py** : decoupage par maturite + statistiques (ATM, skew proxy, term structure).
- **svi\_surface.py** : interpole la variance totale entre tenors calibres.
- **merton\_surface.py** : reevalue Merton sur une grille (k,T) + surface d'erreur absolue.
- **plots.py** : figures matplotlib (smiles) et Plotly (surfaces 3D interactives).

### Amélioration reperee

`surface_to_long_dataframe()` (merton + svi + variante erreur) utilise une double boucle `for i / for j` sur la grille pour construire une liste de dicts. Un seul helper vectorise (masque isfinite + `pd.DataFrame` de colonnes aplaties) remplace les trois copies : 20-50x plus rapide et fin de la triplification.

## 4.10 analysis/ — synthese et rapports

- **`metrics.py`** : table de synthese Heston.
- **`calibration_tables.py`** : tables globales et par tranche (Merton).
- **`merton_diagnostics.py`** : metriques par maturite + stats de poids.
- **`iv_diagnostics.py`** : mark vs mid par zone, RMSE SVI par zone.
- **`svi_metrics.py` / `zones.py`** : zones de moneyness (left/atm/right) et labels.
- **`report.py`** : ecriture des rapports Markdown/CSV.

## 4.11 scripts/ — points d'entree CLI

Orchestration bout-en-bout : `run_svi_calibration.py`, `run_heston_calibration.py`, `run_merton_calibration.py`, `run_smile_analysis.py`, `run_compare_models.py`, `compare_weight_schemes.py`.  
Chaque script fait : `load_snapshot` -> `build_market_panel` -> `calibrate` -> `surfaces` -> `ecriture CSV/figures`. C'est le meilleur endroit pour comprendre l'enchainement reel des appels.



## 5. Le chemin critique du calcul de volatilité

Le 'chemin critique' est la sequence d'appels qui domine le temps CPU. Le situer dit ou une optimisation a un effet reel (et ou elle n'en a pas).

- Etage 1 - `implied_volatility()` sur le panel : execute UNE fois par run, vectorise. Non critique.
- Etage 2 - `scipy.optimize.minimize` : appelle la fonction objectif 200 a 300 fois par calibration. C'est ici que tout se joue, car le cout du pricer sous-jacent est multiplie par le nombre d'iterations.
- SVI : objectif = formule fermee NumPy -> quasi gratuit.
- Merton : objectif = pricer vectorise sur tout le panel en une passe -> rapide meme en global.
- Heston : objectif = boucle Python strike par strike, chaque appel reconstruisant un objet QuantLib + une inversion scalaire -> DE LOIN le poste le plus lourd du depot.

Conclusion : toute seconde gagnee se trouve dans Heston. Les autres modeles sont deja au bon niveau de vectorisation ; les 'optimiser' n'apporterait rien de mesurable.

## 6. Ou et comment optimiser

Recapitulatif priorise. La colonne Impact combine gain de performance et gain de maintenabilite. Les trois premieres lignes sont les seules a fort impact.

| Cible                                   | Probleme                                                            | Action                                                               | Gain                             |
|-----------------------------------------|---------------------------------------------------------------------|----------------------------------------------------------------------|----------------------------------|
| heston/pricer.py :<br>heston_iv_grid    | Moteur QuantLib reconstruit a chaque strike, dans la boucle d'optim | Construire l'engine 1x par tranche, reutiliser pour tous les strikes | 3-8x sur Heston, zero risque     |
| svi + heston :<br>calibrate_all_slices  | Fonction dupquee a l'identique (except silencieux inclus)           | Factoriser dans calibration/runner.py parametre par la fn de tranche | Maintenabilite, 1 seul fix       |
| surfaces :<br>surface_to_long_dataframe | Double boucle for i/for j, triplee (merton IV, err, svi)            | Helper vectorise commun (masque isfinite + colonnes aplaties)        | 20-50x local, fin triplification |
| calibration_weights :<br>_vega_vector   | Boucle itertuples + bs_vega scalaire                                | Remplacer par bs_vega_vec (deja present, vectorise)                  | 10-30x sur la fonction           |
| heston/characteristic.py                | Fonction caracteristique jamais importee (code mort)                | Supprimer, ou finir un pricer Fourier vectorise anti-QuantLib        | Clarte (ou 50x si pricer)        |
| 4x lookup ATM idxmin                    | df.loc[df.log_moneyness.abs().idxmin()] repete 4 fois               | Un helper atm_row(df) partage                                        | Maintenabilite                   |

### Sur Numba / JAX / formats de fichiers

- Numba/JAX : inutile tant que Heston reste sur QuantLib (deja compile en C++). Pertinent seulement si vous ecrivez un pricer Fourier pur Python a accelerer.
- Formats : outputs/ ~ 5,6 Mo de CSV aujourd'hui -> Parquet/HDF5 non justifie. A reconsiderer uniquement si vous empilez des series temporelles de surfaces.
- I/O : load\_snapshot fait une requete par run, pas de goulot. Surveiller list\_snapshot\_dates (GROUP BY plein) seulement si la table grossit beaucoup.

#### Ordre de mise en oeuvre recommande

1) Engine Heston reutilise (gros gain, risque nul). 2) Helper vectorise surface\_to\_long\_dataframe (gain immediat, code plus propre). 3) \_vega\_vector -> bs\_vega\_vec. 4) Factoriser calibrate\_all\_slices + lookup ATM. 5) Trancher le sort de characteristic.py. Chaque etape est independante et testable isolement (la suite tests/ couvre deja les modules concernes).